THE COPPERBELT UNIVERSITY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

SYSTEM DESIGN SPECIFICATION

PROJECT TITLE: THE KEY PERFORMANCE INDICATOR (KPI) EVALUATION SYSTEM

PROJECT SUPERVISOR: DR. DERRICK B. NTALASHA

GROUP MEMBERS

NAMES	**SIN**
COMFORT CHIWELE	22998289
RAYMOSE BANDA	22110688
RUEL MUMBA	22105758
KALENG'A SONEKA	19142153

1. INTRODUCTION

This **System Design Specification (SDS)** provides a comprehensive technical blueprint for the *KPI Evaluation System* developed for the School of ICT at Copperbelt University. It expands upon the Software Requirements Specification (SRS) by detailing the system's architecture, design components, interfaces, data flow, and database structure. This document serves as the primary technical reference for the development team and as a basis for testing and evaluation of the software's design fidelity.

1.1 Purpose

The purpose of this document is to translate both the functional and non-functional requirements of the system into a structured and implementable design. It offers a clear plan for system construction and implementation, enabling developers to build the software, testers to design test cases, and stakeholders to confirm that the design aligns with project goals and end-user needs.

1.2 Scope

This design specification focuses on the internal structure of the web-based KPI evaluation system. The scope includes:

- The overall system architecture and design approach.
- Component descriptions and their interactions.
- Backend database schema and structure.
- Frontend user interface layouts and user flow.
- Technologies and frameworks used in the development (e.g., React, PHP, MySQL).

The SDS does not include deployment strategy beyond the development environment or long-term maintenance strategies.

1.3 Intended Audience

- **Developers**: To implement the design accurately.
- **System Architects**: To review the architectural choices and component interactions.
- **Quality Assurance/Testers**: To guide the preparation of detailed test plans and test cases.
- **Project Supervisors/Stakeholders**: To verify that the design satisfies user and institutional requirements.

2. SYSTEM ARCHITECTURE

2.1. Architectural Style Selection

A **Modular Web-Based N-Tier Architecture** has been selected for the KPI Automation System. The design follows the standard separation of concerns model, dividing the application into three main layers: the presentation layer (frontend), the application logic layer (backend), and the data layer (database).

Rationale:

- **Maintainability and Scalability**: By separating concerns, updates to one layer (e.g., UI changes) do not disrupt the backend or database logic, allowing easier maintenance and future expansion (e.g., mobile access or departmental scaling).
- **Security and Access Control**: Role-based access (admin, supervisor, lecturer) is best managed in a backend logic layer separate from the frontend.
- **Modular Development**: Components such as KPI creation, lecturer evaluation, and reporting are independently designed and can be worked on in parallel.
- **Database-Centric Workflow**: The core of the system is driven by structured data interactions involving KPI records, lecturer data, performance logs, and evaluation results.

An alternative such as a monolithic design was considered but ultimately set aside due to the need for role-specific interfaces and future integration possibilities. A modular n-tier design offers greater long-term flexibility and testability.

2.2 Component Diagram

The system is logically divided in to three distinct layers each with specific responsibilities.

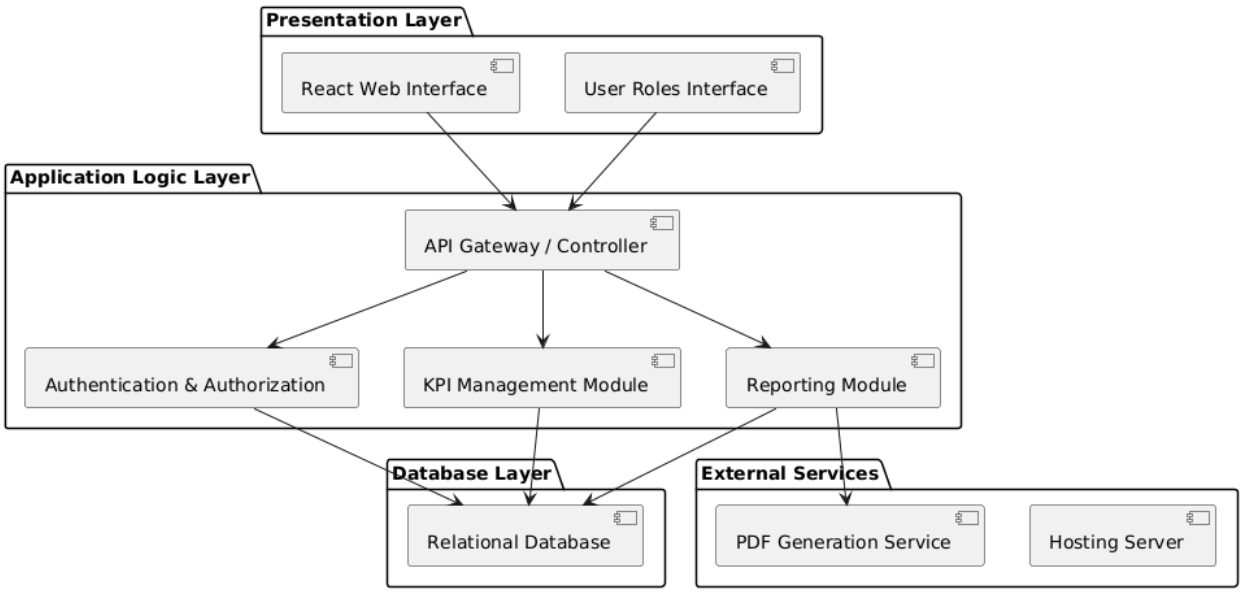

Figure 1: *Component Diagram*

2.2.1 Client Side (Front-End)

The client side is built using:

- **Technology**: React.js (with React Router & Chart libraries)
- **Users**: Admins, Supervisors & Lecturers
- **Functions**:
    - Secure Login (role-based access)
    - Dashboard views (School Overview, Lecturer KPIs, Workplan submission)
    - KPI creation & assignment (Supervisor)
    - Real-time visualization (graphs, charts)
    - PDF report export

2.2.2 Server Side (Back-End)

- **Technology**: ASP.NET Core C#
- **Functions**:
    - Authentication
    - Role & permissions logic
    - Business logic for KPI assignment, evaluation, and reporting

2.2.3 Database Layer

- **Technology**: MySQL
- **Tables**:
    - Users (Lecturers, Supervisors, Admins)
    - Departments
    - KPIs (General & Specific)
    - Workplans
    - Evaluations
    - Reports

2.2.4 External Systems & Services

- PDF Generation Service: Converts UI reports to downloadable PDF files.
- Email Notification Module (optional): Sends alerts/reminders to lecturers and supervisors.
- Hosting Server / Deployment Environment: Cloud or institutional server for live deployment.

2.2.5 Interactions

- All components are connected logically:
    - The UI triggers actions,
    - The logic layer processes requests and applies business rules,
    - The database stores and retrieves data,
    - Optional services enhance functionality (e.g., PDF generation).

2.3 Component Breakdown

The system is composed of modular components, each responsible for handling specific functionalities within the KPI Automation platform. These components work together to ensure a smooth, role-based experience for administrators, supervisors, and lecturers.

- **UI Component (React + Bootstrap + CSS Modules)**: Handles the presentation layer of the system, including the dashboard, charts, forms, modals, and navigation. Built using React and styled with Materialize CSS, this component ensures responsiveness, accessibility, and user interaction across all supported devices. It adapts dynamically based on user roles (e.g., supervisor vs. lecturer).
- **Business Logic Component (ASP.NET Core C# Backend)**: Processes user requests from the frontend, handles logic for KPI creation, assignment, submission of workplans, and performance evaluations. This layer is also responsible for managing role-based access control, ensuring that each user can only access data and actions appropriate to their role.
- **Database Interaction Component (MySQL)**: Provides secure access to stored data, including KPI definitions, workplans, evaluation results, lecturer profiles, and system logs. This layer uses parameterized queries and follows best practices to prevent SQL injection and maintain data integrity.
- **KPI Management Module**: Enables the creation and assignment of KPIs. Supervisors can define general or specific KPIs, link them to one or more lecturers, and categorize them based on departments or timeframes. The module includes dropdown selections and validation logic to ensure accurate data input.
- **Reporting & Visualization Component (Chart.js + PDF Export)**: Generates real-time visual insights for administrators and supervisors. Uses bar and pie charts for performance summaries and provides printable/exportable PDF reports. Visibility of export buttons is dynamically controlled to avoid UI clutter in the final documents.
- **Authentication & Authorization Component (PHP Sessions + Middleware)**: Ensures that only authenticated users can access the system. Role-based middleware restricts access to sensitive pages, such as KPI assignment or lecturer evaluations, based on user privileges (e.g., admin, supervisor, lecturer).

3. DETAILED DESIGN

3.1. Sequence Diagram: Component Recognition

This diagram illustrates the real-time interaction sequence when a user interacts with the system.

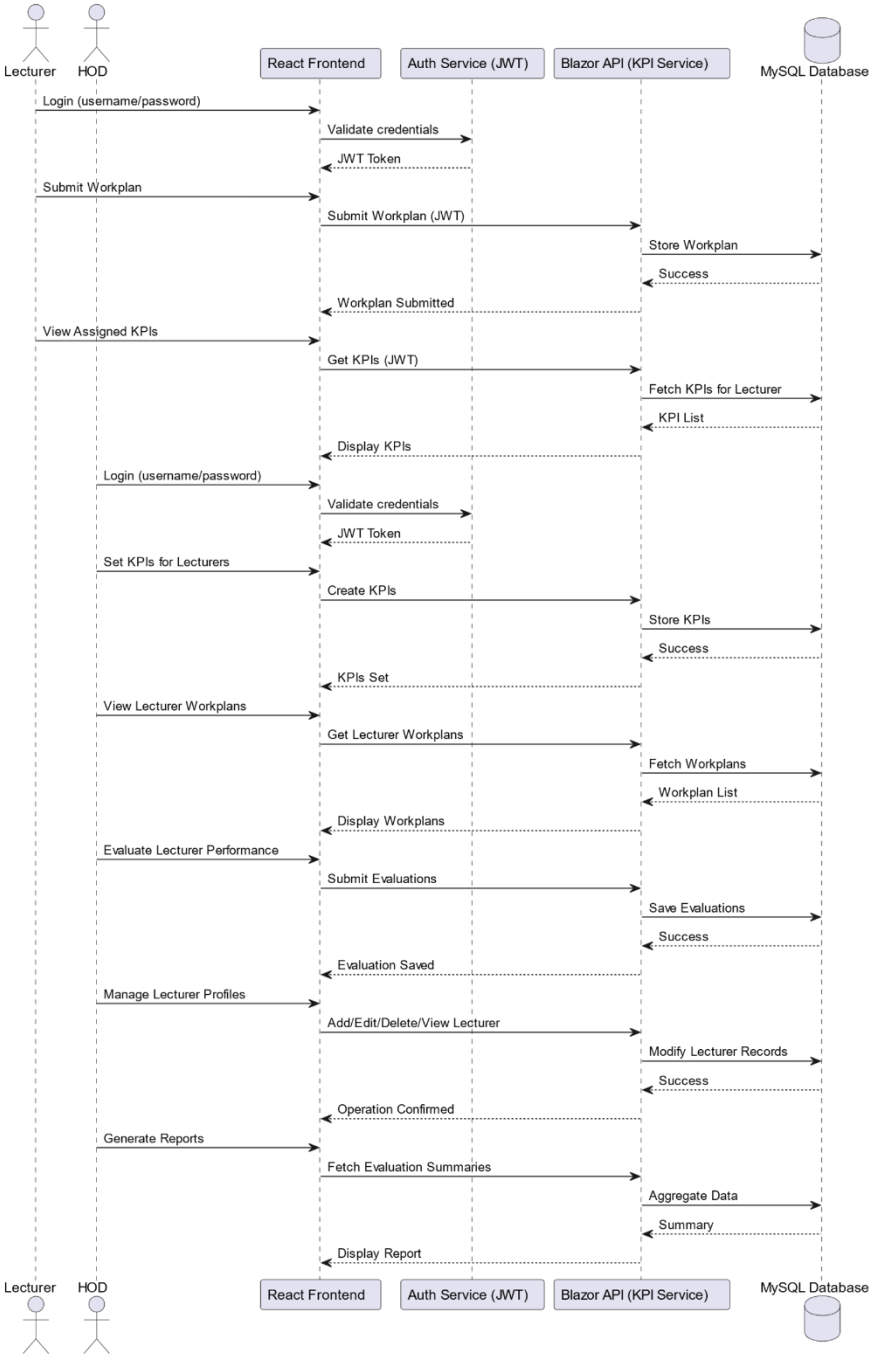

Figure 2: *Sequence Diagram for Component Recognition and Overlay*

3.2. Data Flow

This Data Flow Diagram (DFD) shows the movement of data through the system.

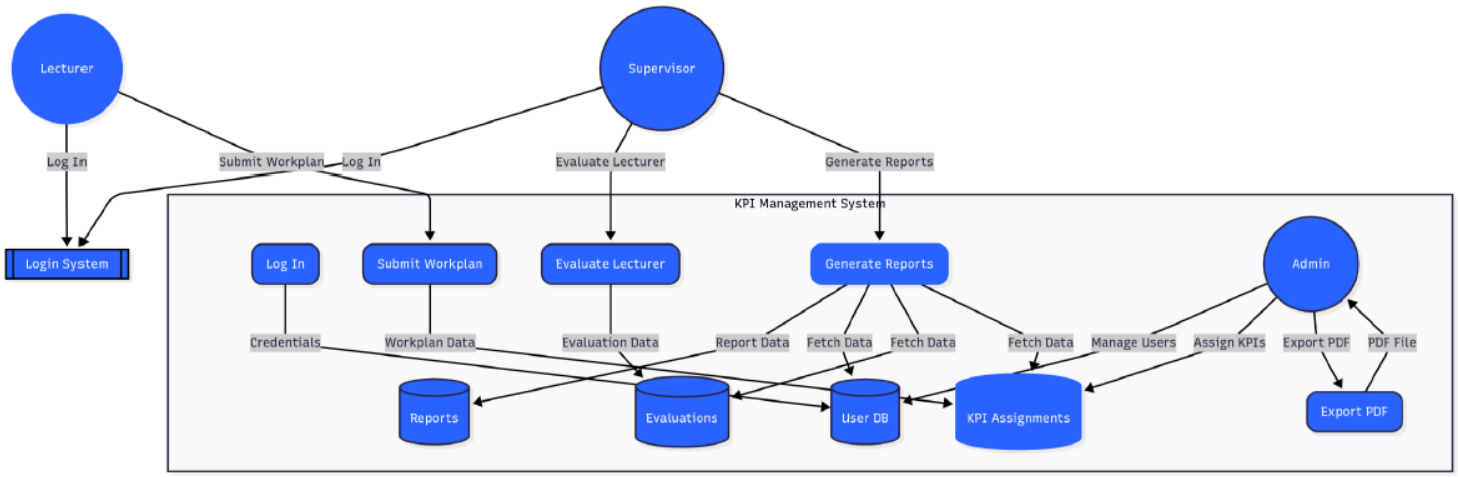

Figure 3: *Data Flow Diagram (DFD)*

4. DATABASE DESIGN

4.1. Schema

The database schema is structured to store and manage information about universities and students, including tables for university profiles, ICT programs, student preferences, and matching results.

4.2. Data Storage

A relational database, such as SQLite, is used to store structured data efficiently.

4.3. Relationships

The schema establishes relationships between tables, linking student preferences with relevant university programs through foreign keys.

4.4. Entity Relationship Diagram

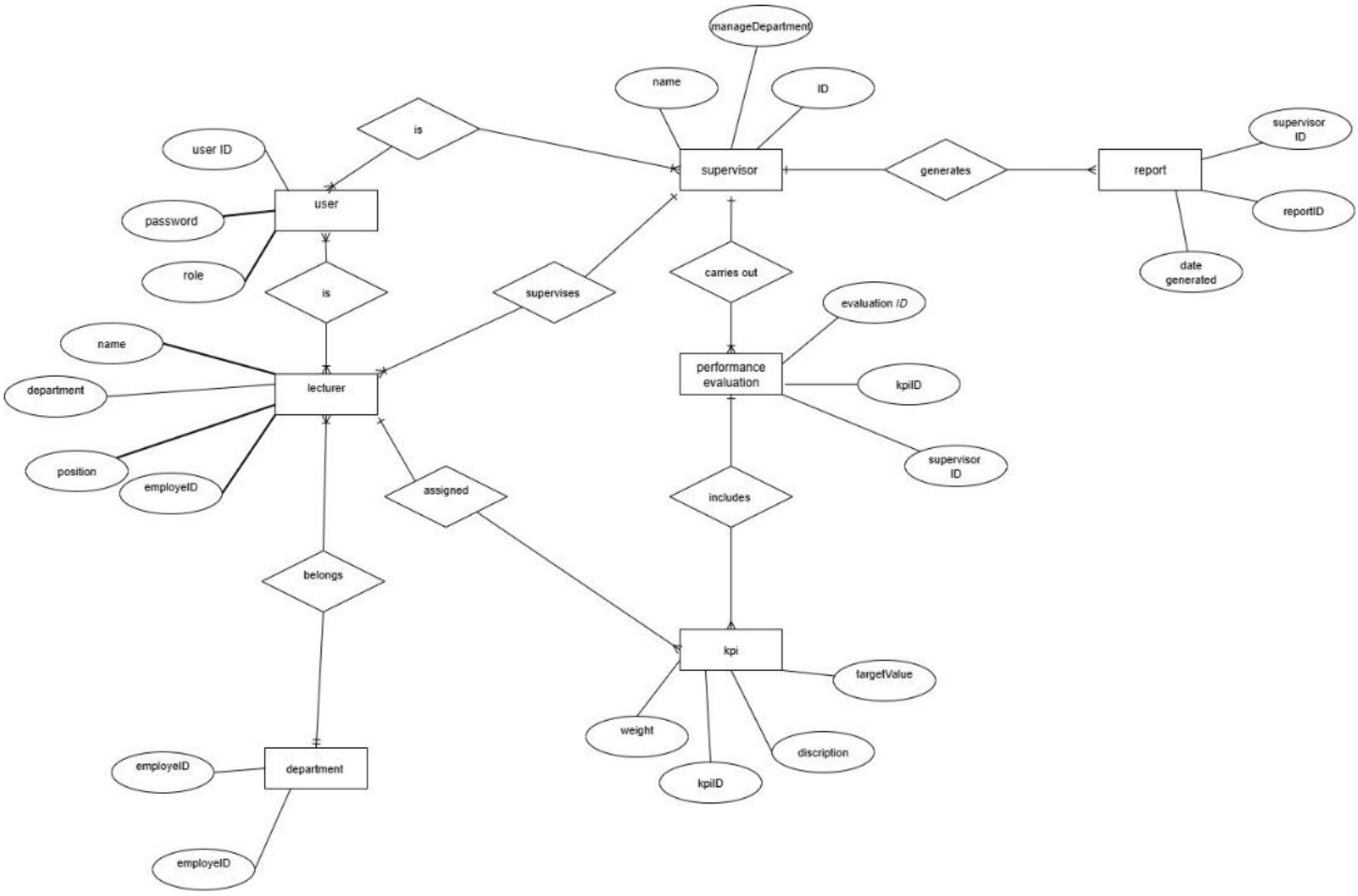

***Figure 4**: Entity-Relationship Diagram*

5. USER INTERFACE (UI) DESIGN

5.1. Design Philosophy

The KPI Automation System adopts a **clean, role-based, and task-focused interface**. The design philosophy emphasizes clarity, responsiveness, and accessibility for users with different responsibilities (e.g., lecturers, supervisors, administrators).

- **Minimalism**: The UI avoids clutter by displaying only relevant content and actions based on user role.
- **Consistency**: Use of consistent layout, button styles, colors (#11486B as a primary brand color), and typography across all pages for a professional look.
- **Responsiveness**: The system is built to function smoothly across various screen sizes, including desktop and tablet devices.
- **User Guidance**: Modals, tooltips, and toast notifications provide feedback and guide users during KPI submission, evaluation, and report export.
- **Visual Insights**: Performance data is presented using interactive charts and summary cards to support quick understanding and decision-making.

6. TECHNOLOGY STACK

The following table outlines the technologies used in the system and the rationale for their selection:

Component	Technology	Justification
Frontend Framework	React.js + Bootstrap	Enables modular UI development with component reuse. Materialize provides a modern, responsive design system with minimal setup.
Backend Server	ASP.NET Core C#	A well-supported, open-source scripting language suitable for building dynamic web applications. Used to handle requests, session management, and logic.
Database	MySQL	A widely adopted, reliable relational database used to store KPIs, user data, workplans, evaluations, and system logs securely and efficiently.
Charting Library	Chart.js	Allows dynamic rendering of bar and pie charts for performance reports with minimal code and good browser support.
PDF Export	jsPDF + html2canvas	Enables generation of downloadable performance reports directly from HTML content.
Routing	React Router DOM	Facilitates client-side routing for smooth navigation between views like Dashboard, Lecturer List, and KPI Pages.
State Management	React State Hooks	Efficient and native approach for managing state locally across functional components, enabling UI reactivity without added libraries.
Authentication & Access	PHP Sessions + Middleware	Ensures role-based secure access, session tracking, and restriction of resources to appropriate users only.

7. NON-FUNCTIONAL DESIGN CONSIDERATIONS

7.1. Performance and Responsiveness

The system is designed to deliver a **smooth and fast user experience** for all users, even during high data loads (e.g., evaluating many lecturers or generating performance charts).

- **Design**:
  - AJAX and React state updates are used to avoid full-page reloads and ensure responsiveness.
  - Pagination is implemented for lecturer lists to reduce UI load time.
  - Backend endpoints are optimized for minimal latency through indexed database queries and lightweight JSON responses.

7.2. Security

Security is critical due to the presence of confidential staff performance data.

- **Design**:
  - Role-based access control is enforced using PHP session validation and route guards in React.
  - Input validation and prepared statements are used on the backend to prevent SQL injection and cross-site scripting (XSS).
  - User sessions time out after a set period of inactivity.
  - Passwords (if applicable) are stored using hashing (e.g., bcrypt).
  - AES-256 encryption can be applied to sensitive data at rest for compliance with data protection laws.

7.3. Error Handling

The system provides clear and helpful error feedback to ensure usability even when issues arise.

- **Design**:
  - If data fails to load (e.g., missing report or API failure), a user-friendly message is displayed with a retry option.
  - For restricted access attempts, users are redirected to the login screen with appropriate feedback.
  - Server-side errors are logged for administrative review without exposing internal errors to users.
  - When exporting or generating charts fails, users are notified via toast alerts, and fallback instructions are provided.